

🚀 3회차 심화 가이드 — 오늘 산 도구, 더 우려먹기

오늘 **superpowers**를 설치하고 **brainstorming·TDD**까지 써봤죠. 여기서 그 상자 안의 나머지 스킬과, 한 단계 위의 도구 두 개를 소개합니다.

다음 회차 스포일러(디자인·배포)는 없어요 — 이미 가진 걸 더 깊이 쓰는 데 집중합니다. 😊

⚠️ **오늘 배운 습관 그대로**: 뭐든 설치 전에 누가 · 몇 명 쓰나 · 살아있나 · 라이선스 4점검. 아래 도구도 예외 없이, 설치 전 AI에게 점검부터 시키세요.

💡 실행은 오늘과 똑같이 **Claude Desktop** 앱 **Code** 탭에서.

1. superpowers, 오늘 안 연 나머지 스킬

superpowers는 스킬 여러 개가 든 상자였죠. 오늘은 brainstorming·writing-plans·TDD·리뷰만 썼어요. **나머지도 이미 여러분 것입니다** — 따로 설치할 것 없이, 상황이 되면 알아서 불러 나옵니다.

스킬	뭘 하나	여러분 사업에서 언제
systematic-debugging	버그 원인을 단계적으로 추적	AI가 "고쳤어요" 했는데 또 터질 때 — 찍지 말고 원인부터
writing-skills	나만의 스킬을 만드는 법	2회차에서 만든 <code>/check</code> 처럼, 반복 작업을 내 스킬로
subagent-driven-development	큰 작업을 나눠 여러 AI가 나눠 처리	기능이 많아질 때 한 번에 몰지 않고
verification-before-completion	"끝났다" 전에 진짜 되는지 확인	오늘 배운 "직접 검수"를 습관으로

💡 뭐가 들었는지 궁금하면 그냥 물어보세요: `"superpowers에 어떤 스킬들이 있는지, 각각 언제 쓰는지 알려줘"`

2. gstack — YC 대표가 직접 쓰는 Claude 셋업

Garry Tan(Y Combinator 대표)이 자기가 쓰려고 만들어 공개한 도구 모음이에요. Claude Code를 **가상 팀처럼** 만들어줍니다 — 역할별로 나눠서.

- **역할 도구**: `/office-hours` (전략 질문 먼저) · `/plan-ceo-review` (사업 관점에서 재검토) · `/review` (버그 잡기) · `/qa` (브라우저로 실제 테스트) · `/ship` (배포) · `/cso` (보안 감사) · `/investigate` (원인 추적)
- **안전 가드레일** (창업가에게 특히 유용): `/careful` (위험한 명령 전 경고 — `rm -rf` 같은 것) · `/freeze` (실수로 뚫 파일 못 건드리게 잠금) · `/guard` (둘 다 켜기)

왜 여러분에게: 오늘 배운 "구상 → 발사 → 검수"를 **역할별 명령으로** 더 촘촘하게 굴릴 수 있어요. 특히 `/careful`은 **비전공자 안전벨트**입니다.

- 🔍 **설치 전 4점검** (오늘 배운 그대로): github.com/garrytan/gstack 에서
 - 누가 → Garry Tan (YC 대표, 실명) · 라이선스 → MIT (자유롭게 사용)
 - 몇 명 쓰나 · 최근에 갱신했나 → 여러분이 직접 눈으로 확인해보세요 (그게 점검!)
 - 그다음: `"github.com/garrytan/gstack 설치 전에 기능 요약하고 보안 점검부터 해줘. 아직 설치는 말고"`

⚠️ gstack은 `~/claude` 설정을 꽤 건드리고, superpowers와 명령이 겹칠 수도 있어요. **점검 보고를 보고** 설치할지 판단하세요. (안 겹치게 쓰는 법도 AI에게 물어보면 됩니다.)

3. awesome-claude-code — 더 찾고 싶을 때 한 곳

Claude Code용 스킬·플러그인·MCP를 모아둔 **큐레이션 목록**이에요 (관리: hesreallyhim). "뭐 더 좋은 거 없나" 싶을 때 여기부터 훑으면 됩니다.

- 링크: github.com/hesreallyhim/awesome-claude-code
- 한국어로 정리된 것도 있어요: github.com/subinium/awesome-claude-code

⚠️ 목록에 있다고 다 좋은 건 아니에요. 오늘 배운 **4점검**으로 골라서 하나씩. 목록은 출발점이지 보증서가 아닙니다.

4. 새 도구 고르는 법 (오늘의 핵심, 계속 통합)

도구는 계속 새로 나와요. **고르는 눈**이 진짜 자산입니다.

- **4점검**: 누가 만들었나 · 몇 명 쓰나 · 살아있나(최근 갱신) · 라이선스
- **설치 전 AI에게 점검 시키기**: `"이거 설치 전에 기능·보안 점검부터. 아직 설치는 말고"`
- **검수는 여전히 내 몫**: 도구가 늘어도 **직접 돌려보고 눈으로 확인**하는 건 안 바뀝니다.

5. 혼자 해볼 미션

- [] superpowers `systematic-debugging` 으로, 예전에 막혔던 버그 하나 다시 추적해보기
- [] 반복하는 내 작업 하나를 `writing-skills` 로 내 스킬로 만들어보기
- [] gstack 저장소를 열어 **4점검** 직접 해보고, 설치할지 말지 스스로 판단
- [] awesome-claude-code에서 내 사업에 쓸 만한 것 하나 찾아 4점검

6. 더 배울 거리

- superpowers : github.com/obra/superpowers
- gstack : github.com/garrytan/gstack
- awesome-claude-code : github.com/hesreallyhim/awesome-claude-code
- 막히면 언제나 → `"이거 최신 방법으로 어떻게 하는지 점검해서 알려줘"`

오즈코딩스쿨 7기 열정반 특강 · 3회차 심화 가이드 · 산 도구를 우려먹는 사람이 멀리 갑니다